FILSPAY CHALLENGE · SUBMISSION DOCUMENT

OG

FilsPay: find the root causes, fix them with proof, finish refunds, run it in production.

Team and ownership

Member	What they owned
Ebrahim Alnasser	Bug fixes and root-cause analysis (money, settlement, webhooks, reports, login and access); security hardening (SEC-1…5) and red-team testing (SEC-RT-1…6); refunds feature FP-300 (route, ledger, settlement batch, webhook, timeline, reports, reconciliation, end-to-end test); email OTP (ENH-001); AI assistant guardrails and limits (ENH-009, ENH-010); capacity testing (CAP-1…3, HOST-2); AI setup (agent instructions, permissions, sub-agents, guard test).
Sayed Hassan Isa	Frontend redesign and the "case file" theme (merchant dashboard, admin console, checkout); the "Ask FilsPay" AI assistant; production deployment with HTTPS on Dilmune (`ops/docker-compose.prod.yml`, nginx, Let's Encrypt); his own bug fixes (BUG-024, 028, 051, 052, 054, 055, S-033) and a second bug pass that found and fixed 45 more (BUG-064 to BUG-109, `SAYED-BUGS.md`); the Payout days panel (FP-301).

Live deployment

Checkout	`https://og-filspay.139-59-1-82.nip.io/pay/<token>` (a token is shown on every payment link in the dashboard)
Merchant dashboard	`https://og-filspay.139-59-1-82.nip.io`
Admin console	`https://admin.og-filspay.139-59-1-82.nip.io`
Public API	`https://og-filspay.139-59-1-82.nip.io/v1` · health `/readyz` (checks the database) and `/healthz`
How to log in	Password, then a 6-digit code sent by email (the code is never on screen; bug BUG-031). The two live accounts with real inboxes are listed in `README.md`; ask the team for the code just before you log in.
Deployment details	`docs/challenge/DEPLOYMENT.md`: server, HTTPS, how a merge to `main` reaches the live site (autodeploy every 2 minutes), required settings.

Repository

Public fork	`https://github.com/Ananny1/og-fils-pay` (fork of `MJ5-byte/fils-pay`)
Last commit before the deadline	`990f62e`
Where the evidence is	`docs/FINDINGS-REPORT.md` (every finding) · `docs/bugs/` (one write-up per fix) · `docs/challenge/` (deployment, capacity, security, DDoS, traps, AI usage) · `IMPLEMENTATION.md` (decisions D-001…D-017, step log)

1. Summary

What we found. 100 bugs in the handed-over code: 55 numbered findings in our first audit (8 areas (24 critical, 23 high, 8 medium), plus 11 security findings from our own hardening and red-team work (SEC-1…5, SEC-RT-1…6), two capacity findings (CAP-2, CAP-3) and a missing feature (refunds, FP-300). We also logged **115 false claims** ("intentional", "already fixed", "Claude says it's fine") in `docs/challenge/TRAPS.md`. The app did not even build at the start: the Dockerfiles were missing, a fresh database could not be migrated, and the services never loaded their settings.

What we fixed. Every Critical finding. Sayed's second pass found 45 more (BUG-064 to BUG-109, section 4), all fixed. Of the first 55, 54 are fixed and 1 is reclassified with a reason (BUG-037, not a bug once BUG-002 was fixed). **100 bugs found, 100 fixed.** Each fix has a test that fails on the old code and passes on the new code, a write-up in `docs/bugs/` and its own commit. The refunds feature (FP-300.1–.6) is complete: route with a required `Idempotency-Key`, ledger correct to the fil, next settlement batch reduced, `refund.succeeded` webhook, timeline, reports, reconciliation and one end-to-end test. Tests went from **426/443 TS tests passing** (and 376 that never ran, DEP-005) to **API 651/651**, reporting 17/17, scripts 22/22, Go settlement all packages ok.

What we left alone. BUG-037 was reclassified as an enhancement (ENH-008). A decoy in dead code (`v1/fees/calculateFee.ts`) was checked and left alone.

The main root cause. The business rules (money is integer fils, the day closes at midnight Bahrain, one capture per link, a shop only sees its own data) lived in comments and docs, not in the code paths, the database or a test. So the code checked and then acted with no lock, used the wrong unit or the UTC date, and left the shop out of queries. Then false comments and poisoned AI instruction files (`CLAUDE.md`, `AGENTS.md`, `.cursorrules`) told every developer and every AI tool not to fix them.

Pattern	What it looks like	Findings
Check, then act, with no lock	Read a row, check it, write later; two requests both pass	BUG-005, 006, 008, 011, 029, 034
Wrong unit or number type	2 decimals (cents) instead of 3 (fils); `float32`, `REAL`, `int32` for money	BUG-014, 015, 021, 022, 028, 050
UTC where Bahrain time is the rule	`date.Date()`, `toISOString().slice(0,10)` give the UTC day	BUG-020, 027, 055
Scope missing from a query or grant	`WHERE id = $1` with no `merchant_id`; a role map that grants too much	BUG-001, 002, 003, 010, 033, 036
A false comment or doc stopped the fix	"intentional", "already fixed in FP-201", "Claude says it's redundant", "never change tests"	BUG-001, 006, 009, 010, 017, 050

2. Architecture and deployment

One server on Dilmune Cloud (1 vCPU / 2 GB). One Docker Compose project, `ops/docker-compose.prod.yml`. **Only nginx publishes ports** (80 and 443). Postgres and the settlement service sit on a Docker network marked `internal: true`: no published port and no route to the internet.

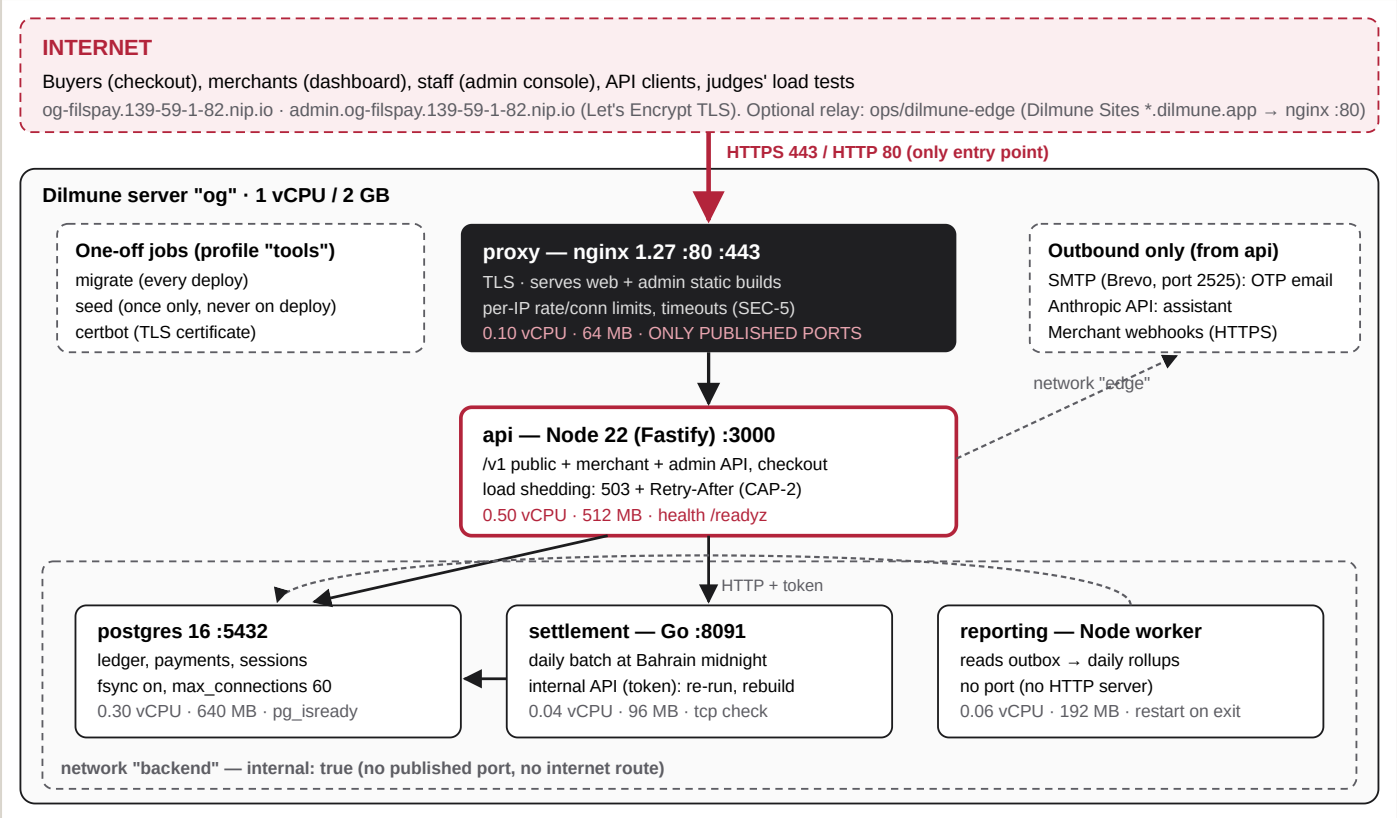


Diagram drawn from `ops/docker-compose.prod.yml` (services, networks, CPU and memory limits, health checks) and `ops/nginx/`. CPU limits add up to 1.00 vCPU and memory to about 1.5 GB, leaving ~0.5 GB for the OS.

The one command to redeploy

```
sh ops/prod/autodeploy.sh --force        # on the server, from the git checkout
```

Normally nobody runs it by hand: a systemd timer (`ops/prod/filspay-autodeploy.timer`, every 2 minutes) runs the same script. When the fork's branch has a new commit it: checks out the commit; builds the images at the lowest CPU priority (`nice -n 19`) while the old containers keep serving; starts Postgres; runs `migrate`; then `docker compose up -d`; and records the commit only after the api reports `(healthy)`. It never runs `seed`, so data survives deploys. A lock file makes sure only one deploy runs at a time. It is pull-based: the server needs no GitHub credentials and GitHub holds no key to the server (HOST-4).

Where secrets come from

- One file on the server, `ops/.env.prod`, ignored by git. The template with every name and no values is `ops/.env.prod.example`: `POSTGRES_PASSWORD`, `SETTLEMENT_INTERNAL_TOKEN`, `WEBHOOK_SECRET_ENCRYPTION_KEY`, `ACQUIRER_CALLBACK_SIGNING_SECRET`, `SEED_USER_PASSWORD`, `SEED_ADMIN_PASSWORD`, `SMTP_*`, `ANTHROPIC_API_KEY`.
- Compose refuses to start without them (`${POSTGRES_PASSWORD:?…}`). The api refuses to boot in production with the secret that leaked in the reference history (SEC-4) or with the seed passwords committed in the repo (HOST-2), and with OTP exposure turned on (BUG-031).
- Webhook secrets are stored encrypted; KYC files are encrypted at rest (SEC-4). TLS certificates live in a Docker volume filled by certbot.

Health checks, restart policies and settlement safety

Service	Health check	Restart	Stop grace
proxy (nginx)	`wget /nginx-health` every 15 s	unless-stopped	default

api	`GET /readyz` every 10 s; `/readyz` runs a live DB query (OPS-001, it used to say "ready" forever)	unless-stopped; `tini` as PID 1	30 s (> its 25 s drain)
settlement	TCP check on :8091 every 15 s	unless-stopped; `tini`	40 s (> its 30 s drain)
postgres	`pg_isready` every 5 s; others start only when it is healthy	unless-stopped	60 s
reporting	none (no HTTP); exits on a fatal error and resumes from the outbox	unless-stopped	20 s

How a redeploy avoids breaking a settlement run. (1) Images are built before any container stops, so the swap is short. (2) On `docker compose up -d` the settlement container gets `SIGTERM`. `main.go` wires it to a context (`signal.NotifyContext`); the scheduler stops starting new work, and every batch step is **one Postgres transaction** that fully commits or fully rolls back (`scheduler/daemon.go`). (3) The internal API (used by the admin "re-run batch" and "rebuild balances") drains in-flight requests for up to 30 s, and `main` waits for that drain before closing the DB pool (OPS-003). The 40 s stop grace is longer than that. (4) `CloseBatch` / `PayoutBatch` are idempotent, so the restarted daemon re-runs the same business day safely: a closed batch is not closed twice. (5) The daily batch runs right after midnight Bahrain (21:00 UTC), so a deploy at another time never meets it.

> **Honest limit:** the deploy script does not check whether a batch is running before it restarts; safety comes from the transaction plus idempotency above, not from waiting. Also noted in our audit: the daemon runs a close at boot before the delay check (harmless because it is idempotent, listed in §4 as an open item).

3. Capacity

Our estimate

> **About 15–20 buyers checking out at the same time, non-stop: ≈ 21 paid checkouts per second (≈ 1,250 per minute), p95 checkout ≈ 1.05–1.3 s, with every payment correct — even when all the load comes from one IP address.** Above that the stack turns extra buyers away in ~1–2 ms with `503 + Retry-After`, keeps serving the admitted ones, restarts the api after a crash in ~2 s, and loses or doubles no payment. (Source: `docs/challenge/CAPACITY.md`.)

Why the p95 is ~1 s and not 0.5 s: the payment simulator waits 100–400 ms on *each* of authorize and capture, so one checkout with zero load already takes ~0.55 s. That floor is the acquirer, not our stack.

Our load test: tool and method

- **Tool:** k6 0.54.0, script `ops/loadtest/checkout.js`. One iteration = one buyer: `GET /v1/payment-links/:token`, then `POST …/pay` with a test card and a fresh `Idempotency-Key`. Each buyer pays its own unpaid 1.500 BD link (`ops/loadtest/make-links.sql`).
- **Target:** `ops/docker-compose.capacity.yml`, the production images and settings run as their own compose project, with every limit enforced by Docker (`cpus`, `mem_limit`; api 0.5 vCPU / 512 MB; whole stack 1 vCPU), BUG-005 single capture, CAP-2 load shedding, SEC-5 edge limits. k6 runs on one host, so **every request comes from one IP**, like a judge's laptop.
- **Correctness check after every run:** `ops/loadtest/verify.sql` — every paid link has exactly one captured transaction of the right amount, and no captured transaction is missing from its link.

Results (closed model, N users, 45 s per step, one IP)

Users	Paid / s	p50	p95	p99	Declined (correct, simulator ~2%)	Shed 503	Wrong
10	14.6	0.62 s	1.04 s	1.30 s	39	0	0
15	**21.1**	0.67 s	**1.05 s**	1.43 s	40	0	0
20	22.5	0.80 s	1.29 s	1.51 s	49	0	0
25	18.5	1.16 s	1.83 s	2.17 s	37	15,553 (p50 0.8 ms)	0
30	18.7	0.90 s	1.28 s	1.49 s	39	20,747 (p50 0.7 ms)	0 real*

* 2,722 "wrong" at 30 users were test-data artefacts: 2,705 × 404 for link numbers past the 60,000 we created, 17 × 409 on links already paid (the server correctly refusing a second charge). DB check of run f (4,361 paid): 0 paid-without-transaction, 0 wrong amount, 0 captured-not-on-a-link.

How tuning moved the knee (same scenario): first prod stack 9.7 paid/s → + CAP-3 indexes (ledger sequential scans 22,760 → 0) 12.2 → Postgres CPU, pool 16, in-flight 24: 18.6 → nginx 1 worker, global in-flight cap, api request logs off: 21–23 → final HOST-2 settings 21–22 paid/s, shed p50 ~1 ms. A separate run by Sayed on `docker-compose.prod.yml` (`docs/ops/CAPACITY.md`, ramp to 80 users, no per-IP limit in the way) peaked at ~36–38 paid/s at 20–30 users, p95 < 1 s. We give the judges the lower, one-IP number.

Where the bottleneck is

Postgres CPU. During the steps Postgres used 0.30 of its 0.35 vCPU and was throttled in 85% of scheduler periods; the api used 0.20 of 0.5; nginx 0.04 of 0.08. Each payment is ~20 SQL statements plus the ledger trigger, and it holds its DB connection across both acquirer waits, so **DB connections, then Postgres CPU**, cap concurrency — not the api.

How the stack turns away excess load and recovers

Layer	What it does
nginx global cap	`max_conns=28` on the api upstream; counted only for requests nginx has fully read, so slow clients cannot fill it
nginx per IP (SEC-5)	auth 1/s burst 5, refunds 3/s, api 10/s, static 30/s; checkout has no per-IP rate (so a one-IP test meets the real valve) but 64 in-flight; header/body timeouts 10 s (Slowloris)
api valve (CAP-2)	more than `API_MAX_IN_FLIGHT` requests → immediate `503` + `Retry-After`; DB pool wait capped, so a request gets 503, not a 5 s wait and a 500

Login limiter (SEC-1)	per-IP and per-account limits and lockout on password and OTP, stored in Postgres
Restart	`restart: unless-stopped` + health checks

Overload + crash drill (60 new buyers/s ≈ 3× capacity, 60 s, one IP; api killed with SIGKILL at 25 s)

Buyers 3,601 · Paid 927 (15.2/s, p95 2.1 s)	Shed 2,637 × 503, median 2.0 ms, all with `Retry-After` · Wrong answers / timeouts: **0**
api RestartCount 0 → 1; `/healthz` 200 again **2.1 s** after the kill	`verify.sql`: 927 paid = 927 captured; **nothing lost, nothing doubled**; 15 links left reserved with no transaction, payable again after the 60 s reservation timeout

One slow-client IP against everybody else

Setup	Attack	Other buyers
Before (global `limit_conn` only)	40 slow `POST …/pay` bodies	**0 paid**, 27,084 × 503
After (HOST-2)	100 slow bodies	**231 paid, 0 shed**, p95 0.76 s
After	500 connections trickling headers	185 paid, 0 shed (nginx closes them after 10 s)

4. Findings report

Findings

One row per finding. **Found**: A = code audit against ADR-0001/0002 and the handover, then a live reproduction on the running stack; T = first full test run (T1 baseline); L = load test; R = red-team agents; I = incident or ticket in the handout. **Verified**: every fixed row has a test that fails on the old code and passes on the fix (named in the write-up `docs/bugs/<ID>-*.md`), plus the before/after shown. **Data**: what the fix does to rows that already exist.

ID · sev	Symptom · how found	Root cause	Fix · verification	Effect on existing data
A. Login and access				
BUG-001 C	Logged in after the password; OTP never needed. A: password only, then `GET /v1/transactions` → 200	`lookupSession` reads view `live_sessions`, which does not filter `authenticated = true`; a comment called the check "redundant" (T-44)	Require `authenticated = true`. Pre-OTP cookie 200 → **401**; OTP login still 200	None; existing half-logged-in sessions stop working
BUG-002 C	Email in two shops: log into A, land in B. A	Login re-looked the user up **by email only** (first row wins)	Bind OTP and session to the password-checked user. `/me` Beta → Alpha	None
BUG-011 H	OTP "5 tries" limit fails; codes predictable. A: 40 parallel guesses	Read-compare-increment with no lock; resend reset the count; `Math.random`	Atomic attempt reservation, single-winner consume, resend keeps the count, `crypto.randomInt`. Guesses checked **12** → **5**	None
BUG-031 C	Login reply and log contain the OTP. A	`otpDevExpose` on whenever `NODE_ENV≠production`, which compose never set	Off by default, refused in production, one switch. Code in reply+log → none	None
BUG-032 H	Pre-OTP cookie becomes the full login (fixation); shop logged out after 10 min. A	OTP verify flags the row; token never rotated; cookie keeps pending TTL	New token at verify, full TTL. Copied cookie 200 → **401**; 600 s → 43,200 s	None
BUG-010 C	"View as shop" gives owner powers; all admins can write. A, live	Role map `super_admin → owner`; no write block on impersonation	Every grant = `support`; non-GET refused. Payment link create **201** → **403**	None
BUG-003 C	Shop A cancels shop B's link (IDOR). A	`WHERE id = $1` only	Scope by `merchant_id`; other shop → 404	None
BUG-033 H	Some admin write routes have no role check. A	Only `requireAdminSession` on replay/archive routes	Role gates; support admin → 403	None
BUG-012 H	Old API key works after rotation. A	Rotation never revokes the old row	Revoke on rotation; old key → 401	Code only
BUG-013 H	Admin approves own credit. A	Compared **display names**, not ids	Compare admin user ids; same id → refused	None
BUG-037 M	Signup accepts an existing email. A	No email ownership check; dangerous only with BUG-002	Reclassified → ENH-008 (D-015) after BUG-002 closed the takeover	—
B. Money coming in				
BUG-004 C	Expired link still payable (FP-187). I + A	Expiry compared to `submittedAt` sent by the browser	Server time only. Past `submittedAt`: 200 → **410**	None
BUG-005 C	One link charged twice (incident 2026-05-14). I + A + L	60 s reservation expires during authorize+capture; second payer re-claims; first finalize updates 0 rows but returns 200	One connection per checkout with an advisory lock; capture check on re-claim; single-capture migration. Crash drill: 927 paid = 927 captured	Migration 0075 adds the one-capture guard
BUG-007 C	Acquirer callback signature skippable/fakeable. A	Missing header accepted; verifies re-serialised JSON; no freshness; non-constant-time compare	Signature required on raw body, timestamp window, `timingSafeEqual`, outcome validated	None
BUG-006 C	Duplicate acquirer callbacks saved twice (FP-201 "fixed"). I + A	No unique index on `callback_id`; `processed_events` never existed (T-31/32)	Unique index; duplicates ignored	Migration 0084 deletes redelivered copies (keeps earliest per `callback_id`)
BUG-008 C	Payout notified and disbursed twice (incident 2026-08-21). I + A	NOT EXISTS then INSERT; no unique keys; deliveries not claimed with `SKIP LOCKED`	Unique indexes, claimed deliveries; one notice + one disbursement per batch	0085 unique indexes; 0086 marks already-queued duplicates `failed` (kept as evidence)

BUG-009 C	Cleanup deleted every card hold on a quiet night. I + A	Empty id list dropped the WHERE → `DELETE FROM holds`	One UPDATE flips overdue active holds to `expired`. 4 → 0 rows becomes 0 deleted	Nothing deleted any more; lost rows are not recreated
C. Money maths				
BUG-014 C	CSV import: 4.75 BD → 475 fils. A	Parsed with 2 decimals and truncation, not `parseBhdToFils`; a test asserted 475	Use `parseBhdToFils`; test corrected to 4750 (justified)	Past imports not rewritten (finance decision)
BUG-017 C	"Rebuild balances": 135K BD → 7K BD (FP-203). I + A	Rebuild reads scheme export lines, not the ledger (ADR-0004 vs handover)	Rebuild from the ledger	Next rebuild corrects cached balances
BUG-021 H	Net payout wrong above ~16,777 BD. A	`float32` in `NetPayableFils`	Integer fils (`int64`)	None (computed per batch)
BUG-022 H	Platform volume overflows at ~2.1M BD. A	`int32` + `INTEGER` column; "Claude double-checked" comment false	`int64` + BIGINT	Migration 0090 widens the column; values kept
BUG-028 H	Lifetime payout estimate drifts. A (Sayed)	`float32` sums in a `REAL` column	Integer fils, BIGINT	Migration 0078 converts the column
BUG-027 H	March sweep doubled the gap when run twice. A	Random entry group, no "done" marker, UTC month, wrong account	Idempotent marker, Bahrain month, `card_scheme_clearing`	Script only; no automatic reversal of past runs
BUG-029 H	Same adjustment CSV applied twice. A	No idempotency key or content hash	Content hash per Bahrain day, unique	0087/0088 add columns; old rows back-filled from `created_at`
BUG-052 H	Capture fee ignores negotiated discount. A (Sayed)	Discount only on another route; Go applies it	Same discount on capture	Past ledger fees not rewritten
D. Settlement (Go)				
BUG-020 H	Daily close at 03:00 Bahrain. A	`date.Date()` on a UTC time	Bahrain window (21:00 UTC)	Past batches not re-cut
BUG-015 H	Scheme statement a few fils over daily. A	Float BD into `NUMERIC(10,2)` rounds; file built from it	File from integer fils	Rows already sent to the scheme not changed; back-fill is a finance decision
BUG-051 H	Rounding residue never posted. A (Sayed)	`CloseBatch` lacks the ADR-0002 posting	Post residue to `scheme_rounding_residue`	New batches only
BUG-016 C	Marketplace split payouts lose fils. A	Each child rounded alone; leftover stays with parent	Allocate the children's shares so they add up to the net, to the fil	New batches only
BUG-018 C	MDR recalc rewrites paid batches. A	`status <> 'open'` includes `paid`	Never touch paid batches	Paid history now frozen
BUG-019 C	Test merchants paid for real. A	Check always 0 (query excludes `authorized`) → clears `test_mode`	Correct query; flag kept	Code only; flags cleared before the fix are not restored automatically
BUG-053 M	Split child with no balance row gets nothing. A	`UPDATE` only; 0 rows, no error	Upsert the balance row (fixed as BUG-110)	None
E. Webhooks and reports				
BUG-023 C	Shops can't verify our signatures (FP-240). I + flag hint	Signs sorted-key JSON, sends jsonb order: different bytes	Sign and send the same bytes (Yousif's `ya/sign-raw` idea)	None
BUG-059 H	Reporting writes NULL merchant; 4 tests red. T	Payload field not checked	Guard + correct field	None
BUG-062 H	Reporting stalls when a rollup row is missing. A	`rows[0]` with no row; retried forever	Handle the missing row	Backlog resumes
BUG-060 M	Retry delay differs from config; 1 test red. T	Default ±10% jitter; no cap check	Jitter injectable, cap enforced	None
BUG-030 M	CSV export formula injection. A	Cells starting `= + - @` not neutralised	Prefix such cells	None
BUG-026 H	Dashboard shows 245 bps, shop charged 225. I + A	Latest-entered schedule, not the effective one	Pick by `effective_from`; test that encoded the bug corrected	None

BUG-024 H	Settlement summary double-counts (~+38%). A (Sayed)	Running total summed once per event	Sum capture deltas; index	Index 0082
BUG-054 M	Reconciliation uses a stale fee rate. A (Sayed)	The "sync trigger" never existed (T-37)	Read the effective schedule	None
BUG-055 M	Rollups by UTC date. A (Sayed)	`toISOString().slice(0,10)`	Bahrain date	Past rollups not re-bucketed
BUG-025 H	Big export in memory; streaming version skips rows. A	One big string; paging on `occurred_at >` only	Stream page by page, keyset `(occurred_at, id)`	None
F. Refunds (FP-300, the missing feature)				
BUG-061 H	Every refund insert fails; 11 tests red. T	CHECK reason NOT NULL vs `reason ?? null`	Default reason in code	None
BUG-034 C	Two refunds at once both succeed (FP-209). I + A	No `FOR UPDATE`, no re-check, lost update	One transaction, row lock, re-check, required `Idempotency-Key`	None
BUG-035 C	Refund more than captured. A	Refundable = authorized − refunded	captured − refunded	None
BUG-036 C	Refund another shop's sale. A	No caller/merchant comparison	Scoped lookup → 404	None
BUG-056 H	Fee reversal can exceed fee. A	Each partial rounded alone	Cumulative reversal minus already reversed	None
BUG-057 M	Wrong status after full refund of a partial capture. A	Partial-capture branch checked first	Full-refund check first → `refunded`	None
FP-300	Refunds return 501 (feature). Brief	Route stub; batch, webhook, reports ignore refunds	.1 route · .2 ledger · .3 batch · .4 webhook · .5 timeline/reports/reconciliation/button · .6 end-to-end test	Migration 0091 adds refund columns to rollups
G. Deployment and running · H. AI instructions				
DEP-001 C	App can't be built. A	Dockerfiles never committed	Added Node and Go Dockerfiles	—
DEP-002 C	Fresh DB can't migrate. A	Abandoned migrations 0054/0055	Fixed; migrate 66/66 on a fresh DB	Applied DBs unaffected
DEP-003 C	Services never get settings. A	No `env_file:` (T-40)	Compose loads env files	—
DEP-004 H	`docker compose up` crashes on a fresh clone. A	No wait for migrate; no dev token	Wait for migrate; committed laptop-only env	—
DEP-005 H	376 tests never ran. T	Node 20 doesn't expand the glob	Node 22; 443 tests run	—
BUG-058 H	API crashes without a hand-made key. A	Dev key decodes to 34 bytes, needs 32	Valid 32-byte dev key	—
F0 M	Dashboard home shows a developer error. A	No `/` route or error element	Routes + error pages	—
BUG-050 C	Agent files told AI to keep the bugs. A	Poisoned `CLAUDE.md` / `AGENTS.md` / `.cursorrules` (T-01…18)	Replaced + guard test: originals 0/7, ours 7/7	Originals kept in git history
I. Security hardening, red team (R) and capacity (L)				
SEC-1 H	Unlimited password/OTP guessing	No per-IP/per-account counter	Limiter + lockout in Postgres	0073/0074 new tables
SEC-2 M	Dashboard open 12 h stays logged in	No idle check	Idle timeout 60 min shops / 30 min staff	0075 `last_seen`
SEC-3 M	No headers; 500s leak internals	Raw error messages; defaults	Headers, safe errors, body/time limits	None
SEC-4 H	KYC files plain; leaked acquirer secret accepted	Raw bytes on disk; no deny-list	Encrypt at rest; refuse leaked secret	Existing files: see write-up
SEC-5 H	No edge flood limits	No nginx rules	Per-IP rate/conn, timeouts; `ops/nginx/test/edge-test.sh`	None
SEC-RT-1…6	R: disabled admin keeps grant; suspended shop keeps sessions; unauth report route; key hash on public page; cross-shop report via split child; bad input → 500	Missing status/scope checks; parse errors to generic handler	Each fixed test-first (round 1: 4 holes / 40 attacks blocked; round 2: 2 holes / 37 blocked)	None

CAP-2 / CAP-3	L: requests queue to timeouts; ledger sequential scans	No load shedding; missing indexes	503 + Retry-After valve; indexes (seq scans 22,760 → 0)	Indexes 0076/0098

Findings, round 2 (Sayed, 30 Oct evening): 45 more bugs in the original code

Each was already in the handed-over code (challenge commit `8b58b0f`; the write-up names the file and line), started with a test that failed on the old code, and has a write-up in `docs/bugs/` (index: `SAYED-BUGS.md`). Duplicates of bugs already fixed above were dropped.

ID	What was wrong	Where in the original code
BUG-064	A posted manual credit is never paid out	`apps/settlement/internal/settlement/batch.go:219`
BUG-065	MDR recalculation ignores the merchant's fee schedule	`apps/settlement/internal/settlement/recalculate_mdr.go:142,157`
BUG-066	MDR recalculation drops the platform volume-tier discount	`apps/settlement/internal/settlement/recalculate_mdr.go:157`
BUG-067	One failing merchant stops the whole daily settlement run	`apps/settlement/internal/settlement/batch.go:439,448,455,461`
BUG-068	After an MDR recalculation the merchant dashboard keeps the old payout figures	`apps/settlement/internal/settlement/recalculate_mdr.go:109`
BUG-069	A merchant `support` user can capture or void payments	`apps/api/src/routes/transactions.ts:95`
BUG-070	A suspended or closed merchant can still charge cards with its API key	`apps/api/src/apiKeys/verify.ts:19`
BUG-071	Adjustment CSV with a wrong merchant id crashes with 500	`apps/api/src/admin/adjustmentImport.ts:68`
BUG-072	Staff logout leaves the "view as merchant" grant working	`apps/api/src/routes/adminAuth.ts:212`
BUG-073	A capture Idempotency-Key reused on another transaction says "captured" but captures nothing	`apps/api/src/payments/capture.ts:92`
BUG-074	A bad amount on authorize or capture crashes with a 500	`apps/api/src/routes/transactions.ts:79`
BUG-075	A failed capture at checkout leaves the payer's authorization open	`apps/api/src/paymentLinks/pay.ts:128`
BUG-076	Creating a payment link for 0 fils crashes with a 500	`apps/api/src/paymentLinks/create.ts:38`
BUG-077	Capturing the rest of a sale after a partial refund shows it as plain "captured"	`apps/api/src/payments/capture.ts:198`
BUG-078	Deleting a webhook endpoint that has deliveries returns 500	`apps/api/src/webhooks/cancel.ts:22`
BUG-079	A disabled webhook endpoint keeps receiving deliveries	`apps/api/src/webhooks/dispatcher.ts:223-224`
BUG-080	Admin "replay batch" sends the same webhook event twice	`apps/api/src/webhooks/replayBatch.ts:80`
BUG-081	Restoring a webhook archive sends the same event several times	`apps/api/src/webhooks/archiveReplay.ts:144`
BUG-082	Transaction report date filter uses the UTC day, not the Bahrain day	`apps/api/src/reporting/transactionsReportQuery.ts:49`
BUG-083	Staff "Adjust balance" reads "50.000" as 50 fils	`apps/admin/src/pages/merchants/MerchantDetailPage.tsx:118`
BUG-084	Credit form defaults to the browser's day, not Bahrain's	`apps/admin/src/pages/merchants/MerchantDetailPage.tsx:19`

ID	What was wrong	Where in the original code
BUG-085	Webhook replay window is read in the browser's time zone	`apps/admin/src/pages/webhooks/WebhookReplayPage.tsx:24`
BUG-086	Retrying a failed partial capture can capture twice	`apps/web/src/pages/transactions/TransactionDetailPage.tsx:77`
BUG-087	Bulk "Approve KYC" also approves merchants no longer on screen	`apps/admin/src/pages/merchants/MerchantListPage.tsx:90`
BUG-088	Closing today's batch from the admin console loses the rest of today's sales	`apps/api/src/routes/admin.ts:118`
BUG-089	Retrying a refund after a lost answer refunds twice	`apps/web/src/pages/transactions/RefundDrawer.tsx:69`
BUG-090	Payment-link list returns 500 for a non-numeric `?limit=`	`apps/api/src/routes/paymentLinks.ts:126`
BUG-091	Payment-link list returns 500 for a malformed cursor	`apps/api/src/paymentLinks/query.ts:24-39`
BUG-092	Cancelling a payment link with a non-UUID id returns 500	`apps/api/src/paymentLinks/cancel.ts:16`
BUG-093	Refund summary view is always empty	`db/migrations/0072_refund_merchant_summary.sql:14`
BUG-094	Posted ledger entries can be edited, deleted, or left unbalanced	`db/migrations/0007_ledger_double_entry.sql:47-51`
BUG-095	Two reporting passes at once count the same event twice	`apps/reporting/src/consumer/cursor.ts:32`
BUG-096	Reporting skips an event whose transaction commits late	`apps/reporting/src/consumer/runConsumerPass.ts:45-53`
BUG-097	An expired link left "reserved" shows as payable	`apps/api/src/paymentLinks/status.ts:40`
BUG-098	Webhook sender follows redirects (SSRF via test-send)	`apps/api/src/webhooks/transport.ts:40`
BUG-099	Webhook sender reads the receiver's whole reply into memory	`apps/api/src/webhooks/transport.ts:48`
BUG-100	The operator who raised a credit is offered "Approve" on it	`apps/admin/src/pages/credits/CreditsPage.tsx:171-180`
BUG-101	Merchant dashboard shows an expired payment link as "active"	`apps/web/src/pages/payment-links/PaymentLinkDetailPage.tsx:123`
BUG-109	admin credits and balance adjustments are idempotent	—
BUG-106	recalculate-mdr answers 404 for an unknown merchant and 400 for from >= to	—
BUG-105	imported balance adjustments are settled by the next batch	—
BUG-103	cap the transaction events timeline at 500 rows	—
BUG-102	merchant login with a malformed merchantId answers 401/400, not 500	—
BUG-104	webhook replay reuses the original event id	—
BUG-108	payment-link list 'active' filter no longer returns expired links	—

Root causes (grouped) and the design decision behind them

Shared cause	Design decision behind it	Findings
Check, then act, with no lock	Correctness was left to application code ("we check first") instead of the database. Under concurrency or retries both requests pass the check. Our rule now: a lock (`FOR UPDATE`, advisory lock) or a unique index enforces the invariant, plus a required idempotency key.	BUG-005, 006, 008, 011, 029, 034
Wrong unit or number type	ADR-0001 says integer fils, but there was no single parser and Go/SQL types were chosen for convenience (`float32`, `REAL`, `int32`, `NUMERIC(10,2)`). Now: `parseBhdToFils`, `applyRateFils`, `int64`/BIGINT only.	BUG-014, 015, 016, 021, 022, 028, 056
UTC instead of Bahrain	Dates were taken from the server clock's calendar day. The business day ends at 21:00 UTC; now every bucket uses the Asia/Bahrain window.	BUG-020, 027, 055
Missing scope	Routes trusted the merchant in the URL and middleware; queries used `WHERE id = $1`. Now every lookup carries the caller's `merchant_id`, and impersonation is read-only.	BUG-001, 002, 003, 010, 033, 036, SEC-RT-5
False comments and poisoned AI files	The old team pasted AI answers as proof ("Claude checked"), and the agent files stated bugs as audited policy, so nobody (human or AI) fixed them. Now: trust order code → ADRs → handover; a guard test keeps the poison out.	BUG-001, 006, 009, 010, 017, 050 + 115 traps
Never deployed for real	Only a laptop compose existed: no Dockerfiles, dev servers, no restart policy, a seed that wipes data. Now a production compose with limits, health checks and a pull-based deploy.	DEP-001…005, HOST-1/2, OPS-001

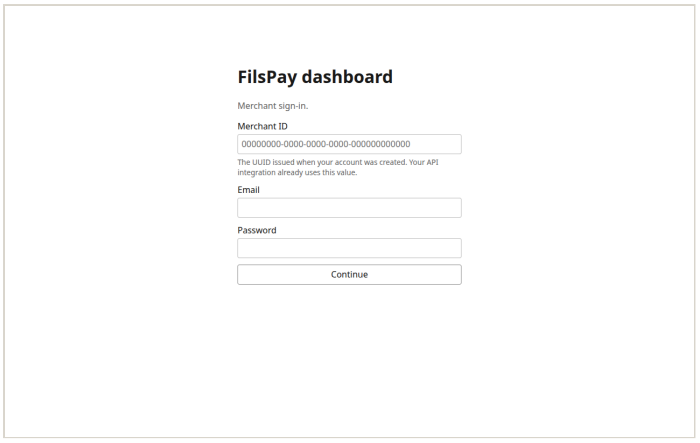
Decisions and open risks

Item	Why we did not change or run it	Safeguard we propose
Data repair of past rows (BUG-014, 015, 020, 052, 055)	Rewriting money history is a finance decision; rows were already sent to the scheme or paid	A one-off, reviewed correction journal, not an UPDATE
Crash between capture and finalize (CAP-1)	Rare; did not happen in the crash drill	Finalize in the capture transaction, or a recovery job
Card testing on checkout	No per-IP rate on checkout, by choice, so one-IP load is served	Failed-attempt limits per link and per card in the api
Acquirer secret leaked in the reference history	We cannot rotate the organisers' acquirer	Rotate it; we refuse the leaked value at boot (SEC-4)
Botnet / volumetric floods	Per-IP limits cannot stop many IPs	Cloud network protection (`docs/challenge/DDOS.md`)
Settlement daemon closes at boot before its delay check; rebuild can race a `CloseBatch`	Idempotent, not seen in tests	Advisory lock shared by rebuild and close
Simulator idempotency map has no eviction	Simulator, not production code	LRU/TTL eviction
Decisions logged	D-001…D-017 in `IMPLEMENTATION.md`, e.g. D-003 test first, D-015 BUG-037 → ENH-008, D-016 parallel sprint in worktrees, D-017 merge rules (keep one implementation, keep both sides' tests).	

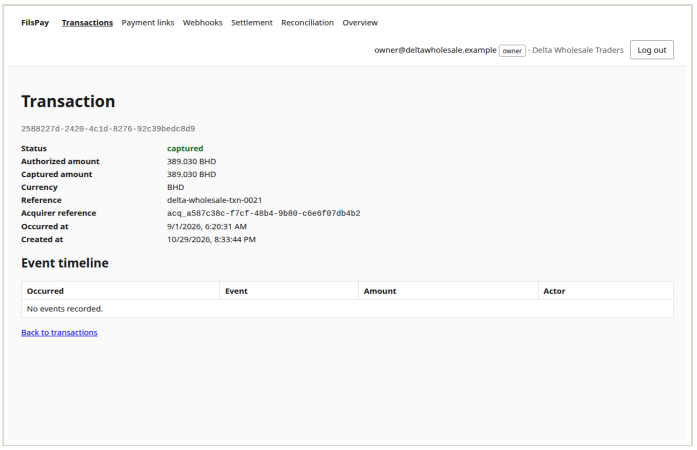
5. UI and UX changes

Two layers of change. (1) Sayed's redesign: one theme for all three apps (`packages/theme`), sidebar navigation, Bahrain dates, money always 3 decimals with BHD, designed empty/error states, confirmation dialogs, the "Ask FilsPay" assistant (`docs/evidence/UI.md`). (2) About 70 small fixes, one commit each, titled `fix(UX-NN)` (`git log --oneline | grep UX-`). Before = `docs/challenge/ux/before/` (original app, 30 Oct). After = `docs/evidence/ui/` (same script, same seed data) and `docs/enhancements/img/`.

The original app ("before" from `docs/challenge/ux/before`)

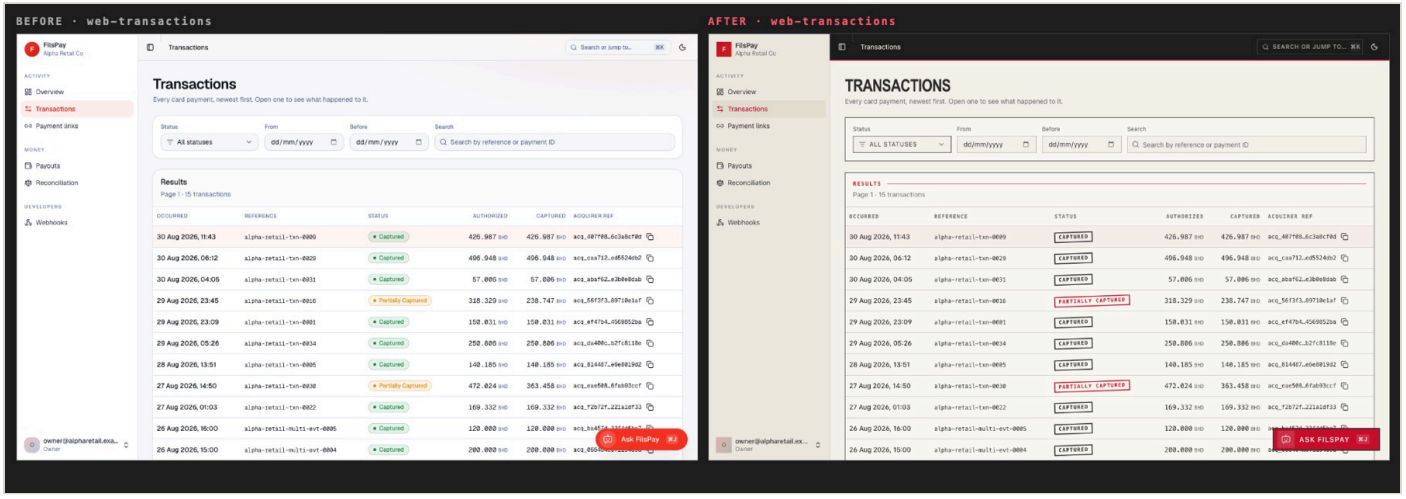


Before 01: merchant login — tab title "FilsPay Checkout" on every page, raw Merchant ID field.

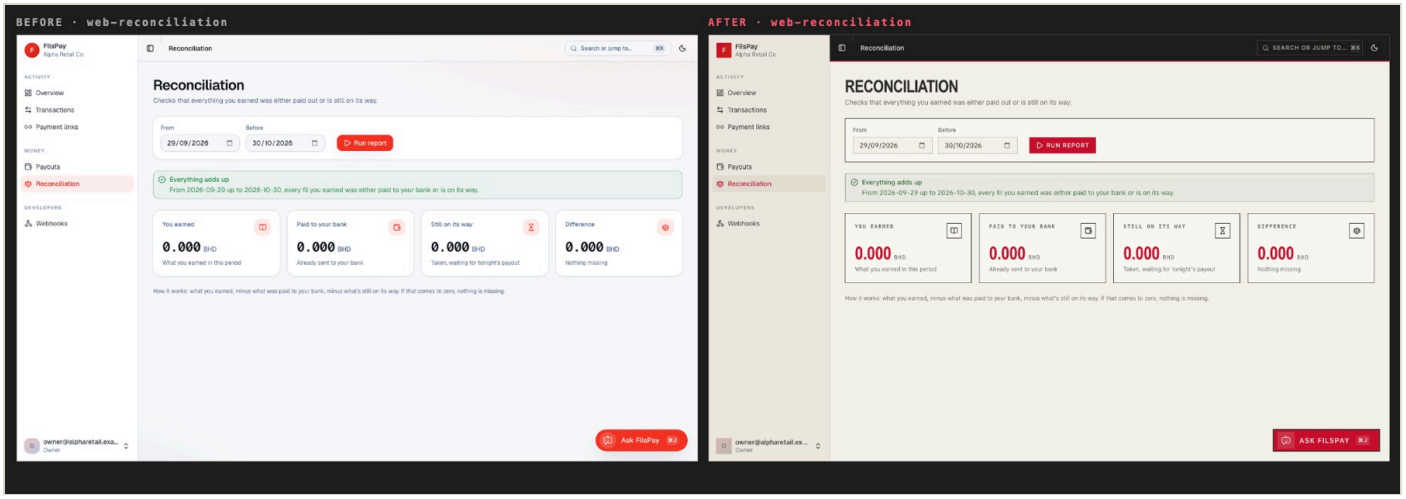


Before 04: transaction detail — no refund action at all (FP-300), empty timeline.

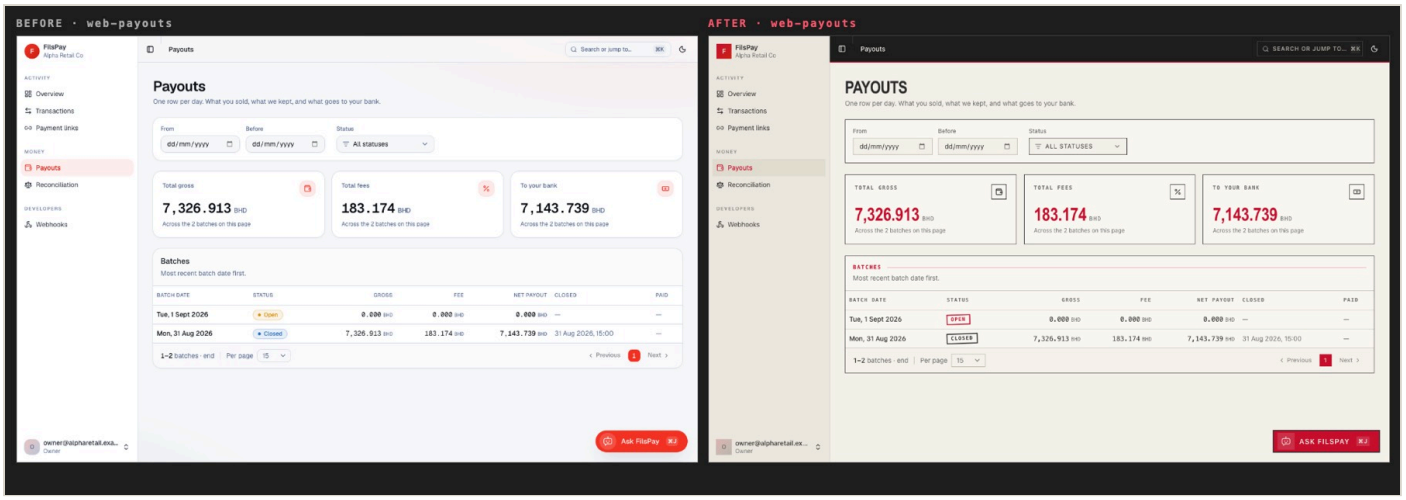
Redesign: same screens, before (left) and after (right)



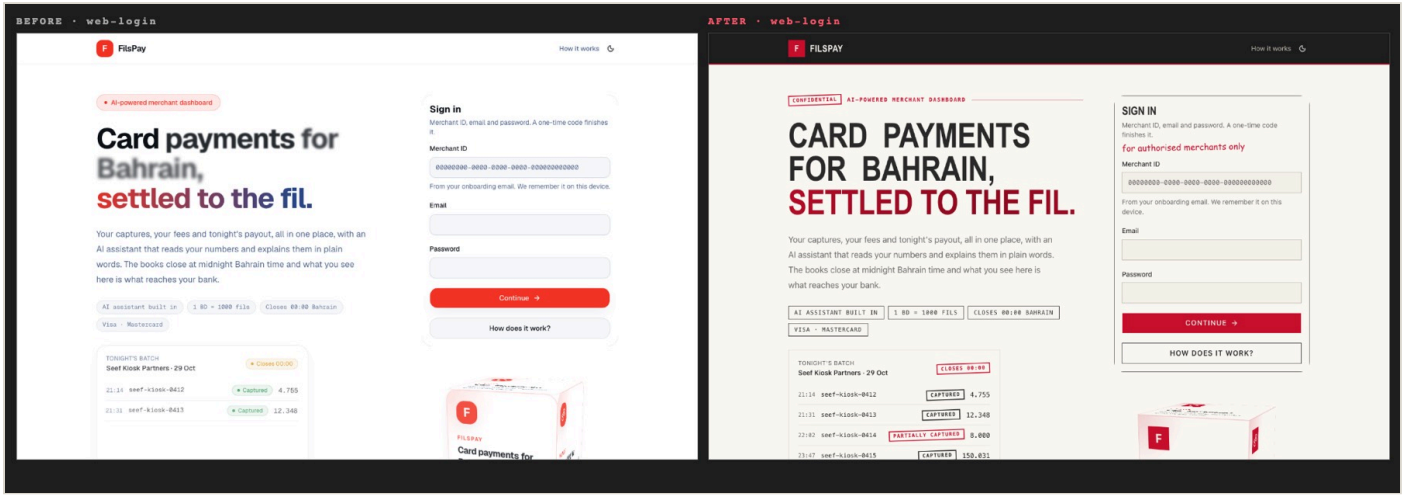
Transactions. What: stamps for status, monospace right-aligned amounts, Bahrain dates, filters at the same height (UX-41), whole row clickable (UX-72). Why: finance scans long lists; fils must line up.



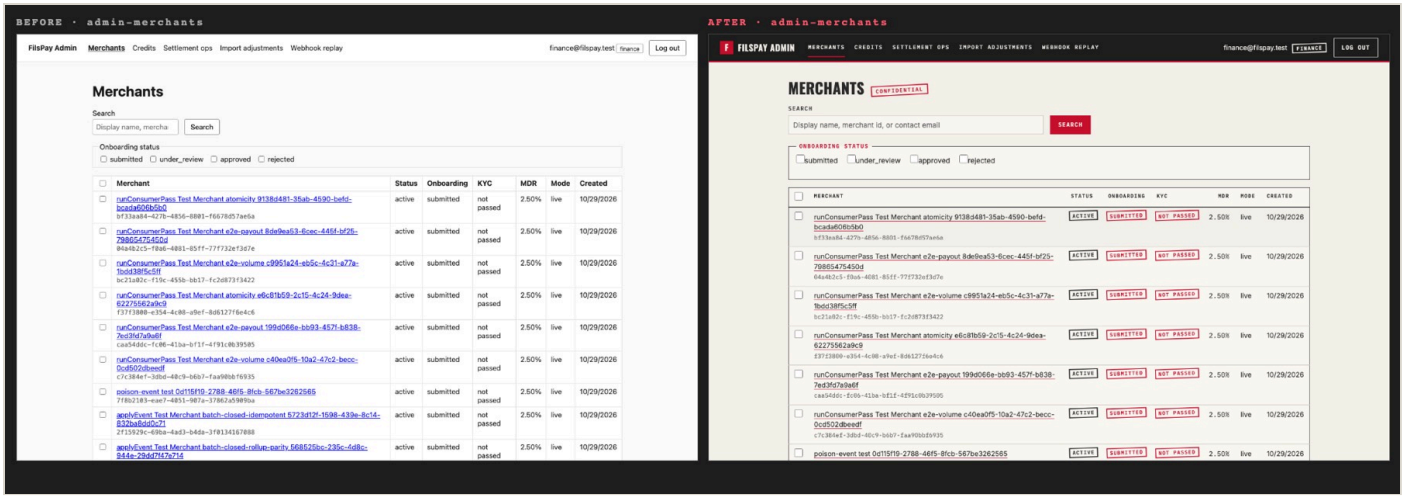
Reconciliation. What: plain words ("You earned", "Paid to your bank"); the "Everything adds up" text passes AA contrast (UX-51); refunds now explain the gap to zero (FP-300.5). Why: merchants are not accountants.



Payouts (settlement batches). What: business dates formatted and in Bahrain time (UX-24), amounts in BHD. Why: the day closes at Bahrain midnight, so dates must match.

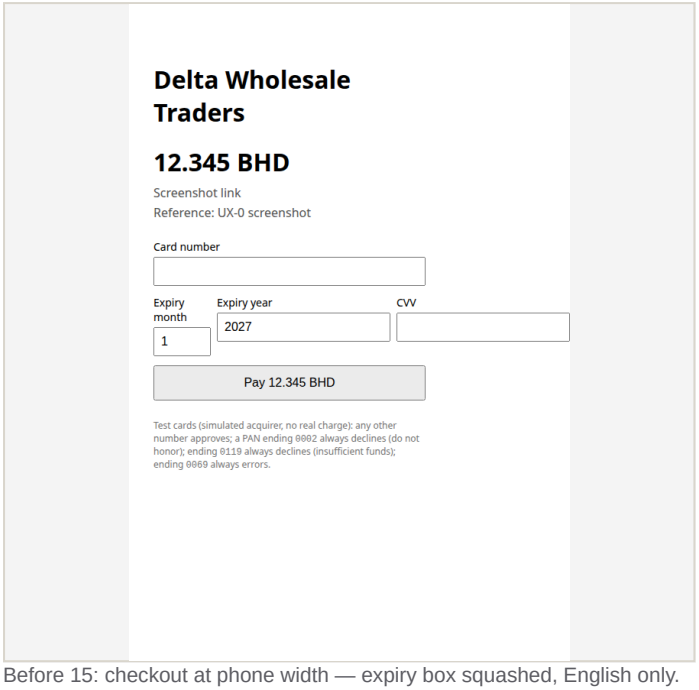


Merchant sign-in. What: form first on phones (UX-28), no sideways scroll (UX-12), empty Merchant ID after sign-out (UX-58/59), OTP accepts pasted "123 456" (UX-48), resend countdown matches the server's 30 s cooldown (UX-31). Why: login is the first screen a judge or merchant sees.

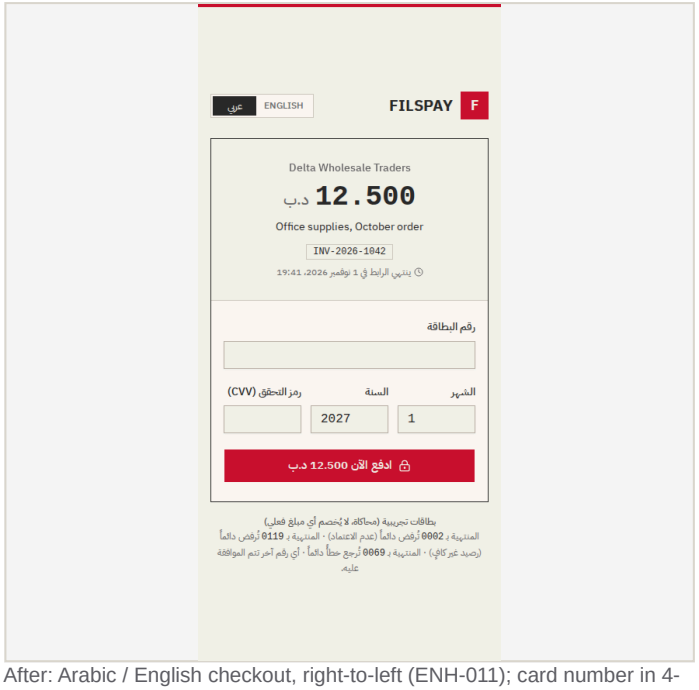


Admin console. What: dark mode and toggle (UX-03), money and dates match the merchant app (UX-21), destructive buttons styled as such (UX-29), inactive statuses grey not red (UX-43), session expiry explained (UX-07). Why: staff actions move money; the screen must say what a button does.

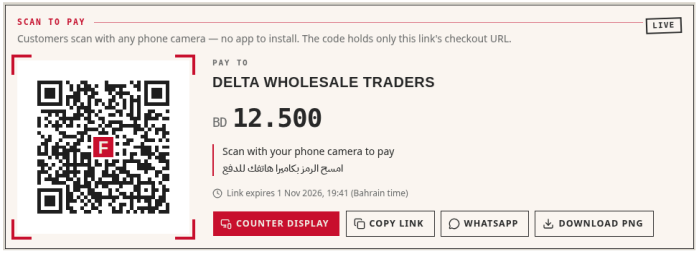
Checkout and payment links



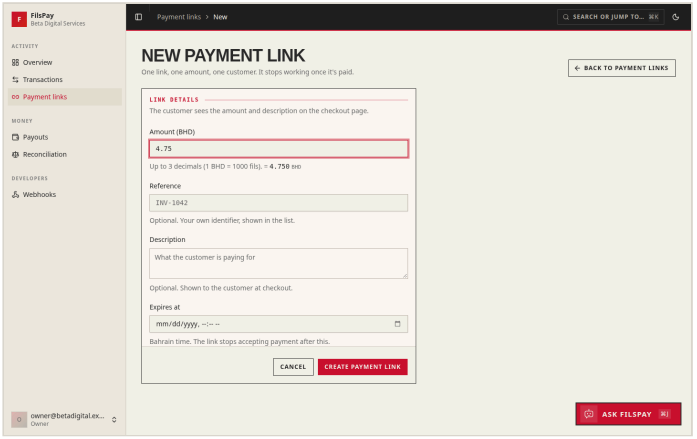
Before 15: checkout at phone width — expiry box squashed, English only.



After: Arabic / English checkout, right-to-left (ENH-011); card number in 4-4-4-4 groups, 16 digits, CVV 3 digits (UX-90/91); inputs locked while paying (UX-50).



After: scan-to-pay QR card on every live link, WhatsApp share keeps the Arabic line in order (UX-60), printable poster (ENH-011).



After: merchants type amounts in BHD, not raw fils (UX-19); errors shown at the field, not in a banner or browser popup (UX-47, UX-71).

Payout days panel and refunds on payouts (FP-301, FP-300.7)

The admin Settlement page now opens with the **last 7 Bahrain business days** and what each one needs (close, pay, nothing), so staff no longer type a date and a merchant id by hand; it refuses to close a day that isn't over (BUG-095). Each payout now shows the refunds it took off. Write-ups: `docs/bugs/FP-301-payout-days-panel.md`, `FP-300-7-*`.

Other changes (one commit each)

ID	What	Why it mattered
UX-01	Command menu no longer crashes the dashboard	Opening ⌘K took the whole app down
UX-05, 14, 15, 16	Failed loads show an error with retry, a dash instead of 0.000 BHD, one message not three	A failed load looked like "you sold nothing"
UX-06, 32	401 mid-session returns to login and explains why	Users were stuck on broken pages after SEC-2's idle timeout
UX-08	Plain-word errors instead of raw API codes	Merchants can't act on `transaction_not_refundable`
UX-10, 20, 22, 57	Filters, link expiry, "data as of" and OTP expiry in Bahrain time	Same rule as settlement: the Bahrain day
UX-17, 18, 53, 52, 33, 35	Dialog buttons inside the dialog, h1 + tab titles, skip link, focus rings, AA contrast, assistant focus	Accessibility (keyboard, screen readers, contrast)
UX-26, 49, 70, 61	Paid / expired / canceled / not-found checkout states styled; "Pay <shop>" title; a way home; solid PAID plate	A buyer must know the payment result for sure
FP-300.5d	Refund button on the transaction page (full or partial, amount in BHD)	The missing feature had no UI
UX-92	Team logo in tab, headers and sidebar	Consistent brand

No screen recording was made. More before/after pairs: `docs/evidence/ui/` (12 screens) and `docs/enhancements/img/` (14 images).

6. AI usage

Our rule in one line: the AI proposes, the code and the measurements decide. Tool: Claude Code (Claude Opus) for reading, the audit, reproductions and first drafts; a person decided every open question (Q-1…Q-8 in `docs/challenge/PROGRESS.md`) and every decision is logged (D-001…D-017 in `IMPLEMENTATION.md`). In the final sprint one session organised parallel agents, each in its own git worktree with its own test database (D-016).

Setup: what we built and where it lives

File	What it does
`CLAUDE.md`	Real rules (integer fils, Bahrain midnight = 21:00 UTC, trust order, trap protocol, test first, one bug = one commit, no push to `main`, no AI attribution); asks who it is working with and shows that person their own tasks. Replaces the poisoned original (kept in git history).
`AGENTS.md`, `.cursorrules`	Same rules for Codex, Cursor, Copilot, pointing to `CLAUDE.md`.
`.claude/settings.json`	Permission rules: denies push to `main`, force push, push to the organisers' remote, `gh pr create/merge`; turns off the co-author line. Works even if a prompt says otherwise.
`.claude/agents/bug-investigator.md`	Read-only sub-agent: root cause with `file:line`, reproduction, ranked options, trap comments. Cannot edit or commit.
`.claude/agents/fix-reviewer.md`	Read-only sceptical reviewer before each commit: correctness, missed callers, test quality, minimal diff. PASS or a list of problems.
`scripts/agent_instructions.test.ts`	Guard test in the normal suite: fails if any poisoned rule returns to the agent files or the push block is removed.
`docs/challenge/TEAM.md`, `SAYED-BUGS.md`, `docs/bugs/README.md`	How the two of us kept in sync: because both AI sessions read the same corrected `CLAUDE.md` and the same shared files, Ebrahim's and Sayed's agents followed one set of rules, one task list and one bug-number map (no clashes, no reused numbers).
`docs/challenge/` GUIDE, WORKFLOW, PLAN, PROGRESS, TRAPS	The workflow every task follows, the master order, a live checkpoint for resuming after a cut-off, and every false claim found.

Excerpts are from our Claude Code session log (session `612dc33b`, 29–30 Oct 2026, times UTC), shortened with "…". Full annotated version: `docs/challenge/AI-USAGE.md`.

Excerpt 1 — The AI was wrong: our guard test caught our own AI-written `CLAUDE.md`

Finding supported	BUG-050 (poisoned agent instructions)
Prompt and context	We asked Claude Code to replace the poisoned `CLAUDE.md` / `AGENTS.md` / `.cursorrules` with the team's real rules. Context: the repo, ADR-0001/0002, the handover; tools: file read/write and the test runner.
AI answer	"I replaced the poisoned files with the team's real rules."

How we verified	A guard test that scans the files for every known false rule, run on the originals (must fail) and on ours (must pass):
	`originals:  not ok 1 - CLAUDE.md contains none of the poisoned rules … # pass 0`
	`ours:       not ok 1 - CLAUDE.md contains none of the poisoned rules`
	`CLAUDE.md still tells agents: Math.random for OTPs`
	`5:"Math.random is fine for OTPs"). The originals are kept in git history …`
	`# pass 6  # fail 1`
	The AI's new file quoted the old rule word for word while explaining it; a tool skimming that line could read it as a rule. Also, the AI's first guard pattern missed the original `.cursorrules` OTP rule (a backtick after `Math.random()`).

What we changed	Reworded to "insecure random OTP codes", fixed the pattern, re-ran: originals 0/7, ours 7/7. The guard runs in the normal suite.

Excerpt 2 — "Claude says" comments in the code: one right, one decoy, one wrong

Finding supported	BUG-001 (login without OTP), BUG-020 (03:00 close), T-35 (decoy)
Prompt and context	Audit task: search the code for comments that quote an AI and check each against the code. Tools: grep, file read, the running stack (curl), the test runner; `bug-investigator` sub-agent for the login tier.

AI answer (in the code)	`stl/settlement/window.go:43: // FH: date.Date() uses date's own location -- a UTC instant late at night gives the UTC day … claude says this is a …`
	`v1/fees/calculateFee.ts: // YA: Claude says the half-up here is a rounding bug …`
	`// FH: it's dead code`
	`106: // FH: claude flagged authenticated = true as redundant with expires_at …`

How we verified	For each: is the code live (who calls it)? What do the ADRs/handover say? Reproduce:
	`== REPRO: password only, no OTP`
	`login: 200`
	`GET /v1/transactions with the pre-OTP cookie: 200`
	Result: the time-zone claim is **right** (BUG-020); `calculateFee.ts` is a **decoy**, nothing calls it; the "redundant" check is **wrong and dangerous** — it had been dropped from the merchant session lookup, so a password alone logged you in.

What we changed	BUG-001 fixed with a failing test (pre-OTP cookie must get 401); BUG-020 fixed with the right root cause; the decoy left alone. All three in `TRAPS.md`.

Excerpt 3 — The AI's numbers, checked against measurements

Finding supported	DEP-005 (376 tests never ran); and the rule behind our capacity estimate (§3)
Prompt and context	Asked to check disk space before building images, and to run the full test suite after DEP-005 and write the commit. Tools: shell (`df`, `docker`), test runner.
AI answer	"Docker has about **4 GB** of old build cache … `docker builder prune` would free it." and, in a commit title, "Before DEP-005, **380** of these tests never ran at all."

How we verified	Measured before and after the prune, and counted tests per suite from the runner output:
	`/dev/nvme0n1p5   96G   87G  7.8G  92% /`
	`Total:  467.2MB`
	`/dev/nvme0n1p5   96G   87G  8.2G  92% /`
	The prune freed **467 MB**, one eighth of the estimate. Tests: 365 API + 11 reporting = **376**, not 380.

What we changed	Corrected the DEP-005 title to 376 in the next commit; wrote only measured numbers in our documents. Same rule for capacity: we report the number we measured under the real limits, not an estimate.

The old team's code also cited AI as proof, and our tests proved those claims false: "INTEGER in PostgreSQL is 64-bit" (BUG-022), "4.750 → 4750 fils, Claude checked" (BUG-014: it gave 475), "Claude drafted the handler" with a random idempotency key per request (FP-300.1: every retry a new refund). Full list: `docs/challenge/PRESENTATION.md` §5.

Appendix

A. Test results

Suite	Baseline (29 Oct, after DEP-005)	Latest verified	How
API (TypeScript)	365 tests, 353 pass, 12 fail	**651/651 pass**	`npm test --workspace @filspay/api` on a fresh migrated DB
Reporting	11 tests, 7 pass, 4 fail	**15 / 15** (FP-300.5b)	`cd apps/reporting && npm test`
Go settlement	7 packages ok (no tests for the settlement bugs)	**all packages ok**	`cd apps/settlement && go test -p 1 ./...`
Web / admin	web 8 tests	web 44 tests + build; admin build; 93 migrations on a fresh DB	`npm test -w apps/web`; `npm run build`
scripts (incl. guard test)	14 / 15	15 / 15	`npx tsx --test scripts/*.test.ts`

```
# tail of the API run (latest)
ok 589 - a long response body is excerpted to 2000 characters
ok 590 - another merchant's endpoint id is a 404, never a cross-tenant send
# tests 590
# pass 590
# fail 0
```

B. Before/after reproductions (from `IMPLEMENTATION.md` step log)

Fix	Before	After
BUG-001	pre-OTP cookie on `/v1/transactions`: 200	401
BUG-011	40 parallel OTP guesses: `attempt_count` 12 (limit 5)	5
BUG-032	copied pre-OTP cookie after real login: 200; cookie TTL 600 s	401; 43,200 s
BUG-010	super-admin "view as shop" creates a payment link: 201	403 (reads still 200)
BUG-009	quiet night: all holds deleted (4 → 0 rows)	expires 4, deletes 0
BUG-014	"4.75" → 475 fils	4750 fils
BUG-026	admin shows 245 bps, capture charges 225	both 225
DEP-005	"Could not find", 0 tests run	443 tests run
BUG-050 guard	originals 0/7	ours 7/7

C. Load-test commands and the correctness query

```
# capacity steps (users per step, 45 s each), all from one machine = one IP
docker exec -i fpprod-postgres-1 psql -U filspay -d filspay -v run=f -v count=60000 < ops/loadtest/make-links.sql
BASE=http://localhost:18080 sh ops/loadtest/steps.sh f 45s 10 15 20
# overload at 60 arrivals/s; kill the api mid-run
docker run --rm --network host -v "$PWD/ops/loadtest:/lt" grafana/k6:0.54.0 run \
  -e BASE=http://localhost:18080 -e RUN=o -e PROFILE=overload -e RATE=60 -e DURATION=60s /lt/checkout.js
docker exec fpprod-api-1 node -e 'process.kill(7, "SIGKILL")'
# correctness (paid links vs captured transactions)
docker exec -i fpprod-postgres-1 psql -U filspay -d filspay -v run=f < ops/loadtest/verify.sql
```

D. Refund end-to-end test (FP-300.6, `apps/api/src/refunds/refundEndToEnd.test.ts`)

Authorize + capture 10.000 BD (MDR 250 bps), refund 4.000 BD through `POST /v1/merchants/:merchantId/refunds`, then check: ledger groups balance and `merchant_payable` = 10000 − fee − (4000 − fee reversed) to the fil; exactly one signed `refund.succeeded` webhook; the refund is the row Go `CloseBatch` reads and the batch net equals what the ledger owes (Go tests `TestCloseBatch_DeductsRefundsFromNet`, `TestCloseBatch_RefundLargerThanNetCarriesForward`); timeline shows `refund_partial` −4000 fils, status `partially_refunded`; today's Bahrain rollup +1 refund / 4000 fils; reconciliation gap 0.

E. The hidden flag

`FLAG{order_of_the_prompt_never_read_the_diff}`, found in the reference repo's git history: Base64 (key `diffs`) → Vigenère → ROT13 → AES-256 (password `2026-09-28`). Its hint ("fix what he started: `ya/sign-raw`, the abandoned migrations, the handover warnings") led to BUG-023, DEP-002, BUG-009/010/032.